

CMSE 801 - Day 3



Calculating savings growth using lists and loops

September 4, 2024

General Course Announcements

- The due dates for Homework assignments and dates for the quizzes will be finalized by next week and updated on the course schedule page.
- Office hours and help room hours have started this week! **You can find the schedules times and location on the course homepage.**
- Remember: at the end of class **you will be uploading your in-class notebook to D2L.**

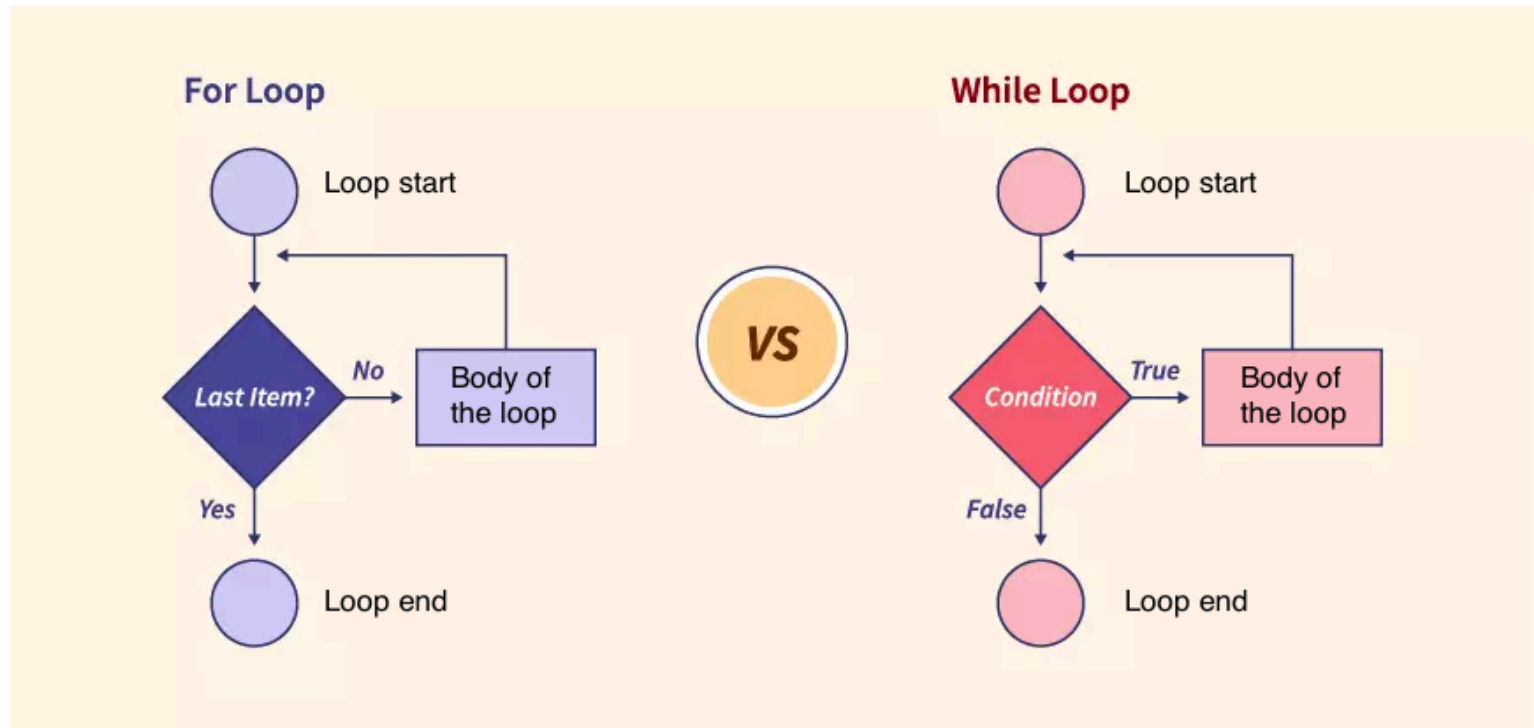
Questions/comments from Day 3 PCA

- Some confusing terminology and jargon coming up here and there. e.g. things "initialize" and "iterate"
- Some challenges with understanding what exactly a question is asking you to do.
- When/why would I use a nested loop?
- What are the key differences between `for` loops and `while` loops?
 - **We'll spend a bit of time talking/thinking about this**
- Some struggles with Question 5 and Question 6
 - **We'll take a look at some possible solutions and some potential pitfalls**

The `for` loop vs. the `while` loop

What's the key difference?

The `for` loop vs. the `while` loop



Let's look at some code examples

```
In [11]: # Loop by integer values or "by index"  
for i in range(0,3):  
    print(i)
```

```
0  
1  
2
```

```
In [13]: # Loop by value in a list  
mylist = ['red', 'yellow', 'blue']  
for value in mylist:  
    print(value)
```

```
red  
yellow  
blue
```

```
In [9]: # Loop until the the "count" value is greater than or equal to 3  
count = 0  
while count < 3:  
    print(count)  
    count += 1
```

0
1
2

Reviewing Question 5

Below is a possible solution to Question 5, will it work? If not, what's missing?

```
In [14]: x = [2,4,6,8,10,12,14,16,18]
y = [10,8.25,7.5,7,6.5,7,7.5,8.25,10]

# Create a loop to add the x and y values together and store them in a list
for i in range(len(x)):
    # index the x and y lists, add their values, and append them to the
    addition.append(x[i] + y[i])

# Check to see if it worked
print(addition)
```


NameError

Traceback (most recent

call last)

Cell In[14], line 7

```
    4 # Create a loop to add the x and y values together and s  
tore them in a list called "addition"
```

```
    5 for i in range(len(x)):
```

```
        6 # index the x and y lists, add their values, and app  
end them to the list
```

```
----> 7     addition.append(x[i] + y[i])
```

```
    9 # Check to see if it worked
```

```
   10 print(addition)
```

NameError: name 'addition' is not defined

This should work!

```
In [15]: x = [2,4,6,8,10,12,14,16,18]
y = [10,8.25,7.5,7,6.5,7,7.5,8.25,10]

# Initialize my empty list
addition = []

# Create a loop to add the x and y values together and store them in a list
for i in range(len(x)):
    # index the x and y lists, add their values, and append them to the list
    addition.append(x[i] + y[i])

# Check to see if it worked
print(addition)
```

[12, 12.25, 13.5, 15, 16.5, 19, 21.5, 24.25, 28]

Reviewing Question 6 (the nested loop!)

What if for each individual `x` value, we want to calculate $(x + y)^2$ for *all* of the `y` values and print the list for each `x` value?

```
In [16]: x = [2,4,6,8,10,12,14,16,18]
y = [10,8.25,7.5,7,6.5,7,7.5,8.25,10]

# Time to nest some loops!
for i in range(len(x)):
    xy_squared = []
    # index the x and y lists, add their values, and append them to the
    for ii in range(len(y)):
        xy_squared.append((x[i] + y[ii])**2)

    # Check to see if it worked
    print(xy_squared)
```

```
[144, 105.0625, 90.25, 81, 72.25, 81, 90.25, 105.0625, 144]
[196, 150.0625, 132.25, 121, 110.25, 121, 132.25, 150.0625, 196]
[256, 203.0625, 182.25, 169, 156.25, 169, 182.25, 203.0625, 256]
[324, 264.0625, 240.25, 225, 210.25, 225, 240.25, 264.0625, 324]
[400, 333.0625, 306.25, 289, 272.25, 289, 306.25, 333.0625, 400]
[484, 410.0625, 380.25, 361, 342.25, 361, 380.25, 410.0625, 484]
[576, 495.0625, 462.25, 441, 420.25, 441, 462.25, 495.0625, 576]
[676, 588.0625, 552.25, 529, 506.25, 529, 552.25, 588.0625, 676]
[784, 689.0625, 650.25, 625, 600.25, 625, 650.25, 689.0625, 784]
```

Goals for today

- Get some practice and comfort with writing loops and using lists in Python.
- Use these tools to track accumulated savings over time using a retirement investment model.

Looking forward:

Exploring conditional statements and a brief intro to functions in the Day 4 PCA

- "If" statements in Python (and their more complex "elif" and "else" statements)
 - Used to control code flow
- Functions in Python
 - Modular
 - Reusable
 - Useful!

Note: this might also feel like a slightly large PCA, but you have a few days to work through it, I encourage you to tackle it sooner than later.

